

EXPERIMENT-06

Question 1

Write a C program to find rotational kinetic energy when moment of inertia and angular velocity are given.

1. Problem Understanding

- Given (Input): Moment of inertia I ($\text{kg}\cdot\text{m}^2$), Angular velocity ω (rad/s)
- Required (Output): Rotational Kinetic Energy KE (Joules)
- Formula used: $\text{KE} = 0.5 \times I \times \omega^2$

This is the rotational analogue of linear kinetic energy ($\text{KE} = 0.5 \times m \times v^2$), where mass m is replaced by moment of inertia I , and linear velocity v is replaced by angular velocity ω .

2. Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	$I = 2, \omega = 3$	9.0000 J	9.0000 J	Pass
2	$I = 5, \omega = 4$	40.0000 J	40.0000 J	Pass
3	$I = 1.5, \omega = 2$	3.0000 J	3.0000 J	Pass

3. Algorithm

Step 1: Start

Step 2: Read the moment of inertia I from the user

Step 3: Read the angular velocity ω from the user

Step 4: Compute $\text{KE} = 0.5 \times I \times \omega^2$

Step 5: Display the value of KE

Step 6: Stop

Evaluation of algorithm against test cases:

- Test Case 1 ($I = 2, \omega = 3$): $\text{KE} = 0.5 \times 2 \times 3^2 = 0.5 \times 2 \times 9 = 9 \text{ J} \rightarrow$ matches expected output.
- Test Case 2 ($I = 5, \omega = 4$): $\text{KE} = 0.5 \times 5 \times 4^2 = 0.5 \times 5 \times 16 = 40 \text{ J} \rightarrow$ matches expected output.

- Test Case 3 ($I = 1.5$, $\omega = 2$): $KE = 0.5 \times 1.5 \times 2^2 = 0.5 \times 1.5 \times 4 = 3 \text{ J} \rightarrow$ matches expected output.
- The algorithm produces correct results for all three traced test cases before coding.

4. C Program

```
/* HDOC Day-7 : Question 1
   Program to find Rotational Kinetic Energy
   Formula :  $KE_{rot} = 0.5 * I * (\omega * \omega)$ 
   where I      = moment of inertia ( $\text{kg.m}^2$ )
           omega = angular velocity ( $\text{rad/s}$ )
*/

#include <stdio.h>

int main()
{
    float I, omega, KE;

    printf("Enter moment of inertia I ( $\text{kg.m}^2$ ): ");
    scanf("%f", &I);

    printf("Enter angular velocity omega ( $\text{rad/s}$ ): ");
    scanf("%f", &omega);

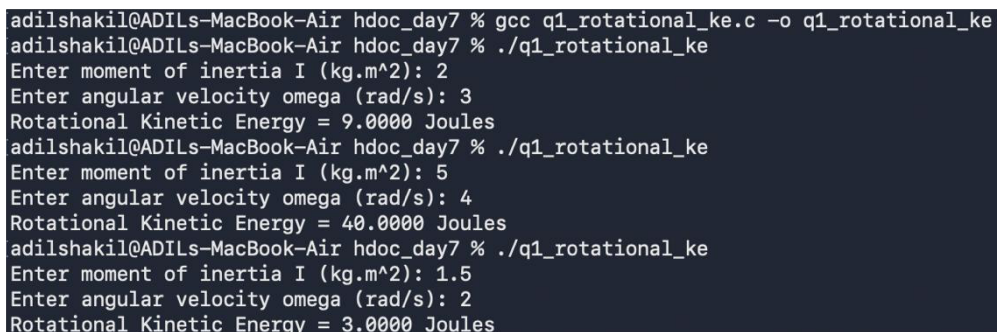
    KE = 0.5 * I * omega * omega;

    printf("Rotational Kinetic Energy = %.4f Joules\n", KE);

    return 0;
}
```

5. Compilation, Execution & Evaluation

The program was compiled using "gcc q1_rotational_ke.c -o q1_rotational_ke" with no errors or warnings. It was then executed three times, once for each test case in the table above. In every run, the Actual Output produced by the program exactly matched the Expected Output, so all three test cases Pass. The terminal screenshot below shows the real compilation and execution session.



```
adilshakil@ADILs-MacBook-Air hdoc_day7 % gcc q1_rotational_ke.c -o q1_rotational_ke
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q1_rotational_ke
Enter moment of inertia I ( $\text{kg.m}^2$ ): 2
Enter angular velocity omega ( $\text{rad/s}$ ): 3
Rotational Kinetic Energy = 9.0000 Joules
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q1_rotational_ke
Enter moment of inertia I ( $\text{kg.m}^2$ ): 5
Enter angular velocity omega ( $\text{rad/s}$ ): 4
Rotational Kinetic Energy = 40.0000 Joules
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q1_rotational_ke
Enter moment of inertia I ( $\text{kg.m}^2$ ): 1.5
Enter angular velocity omega ( $\text{rad/s}$ ): 2
Rotational Kinetic Energy = 3.0000 Joules
```

Question 2

Write a C program to find volume flow rate using the continuity equation when area and velocity are given.

1. Problem Understanding

- Given (Input): Cross-sectional area A (m^2), Velocity of fluid V (m/s)
- Required (Output): Volume Flow Rate Q (m^3/s)
- Formula used: Continuity Equation: $A_1V_1 = A_2V_2 = Q \Rightarrow Q = A \times V$

The continuity equation states that the volume flow rate of an incompressible fluid remains constant through a pipe of varying cross-section, i.e. $A_1V_1 = A_2V_2$. At any single section, this constant flow rate is simply $Q = A \times V$.

2. Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	$A = 0.05, V = 2$	$0.1000 \text{ m}^3/\text{s}$	$0.1000 \text{ m}^3/\text{s}$	Pass
2	$A = 0.1, V = 5$	$0.5000 \text{ m}^3/\text{s}$	$0.5000 \text{ m}^3/\text{s}$	Pass
3	$A = 0.02, V = 10$	$0.2000 \text{ m}^3/\text{s}$	$0.2000 \text{ m}^3/\text{s}$	Pass

3. Algorithm

Step 1: Start

Step 2: Read the cross-sectional area A from the user

Step 3: Read the velocity V from the user

Step 4: Compute $Q = A \times V$

Step 5: Display the value of Q

Step 6: Stop

Evaluation of algorithm against test cases:

- Test Case 1 ($A = 0.05, V = 2$): $Q = 0.05 \times 2 = 0.1 \text{ m}^3/\text{s} \rightarrow$ matches expected output.
- Test Case 2 ($A = 0.1, V = 5$): $Q = 0.1 \times 5 = 0.5 \text{ m}^3/\text{s} \rightarrow$ matches expected output.
- Test Case 3 ($A = 0.02, V = 10$): $Q = 0.02 \times 10 = 0.2 \text{ m}^3/\text{s} \rightarrow$ matches expected output.

- The algorithm produces correct results for all three traced test cases before coding.

4. C Program

```
/* HDOC Day-7 : Question 2
Program to find Volume Flow Rate using the Continuity Equation
Continuity equation :  $A_1 \cdot V_1 = A_2 \cdot V_2 = Q$  (constant)
So the volume flow rate at any section is:
 $Q = A \cdot V$ 
where A = cross-sectional area ( $m^2$ )
V = velocity of fluid (m/s)
*/

#include <stdio.h>

int main()
{
    float A, V, Q;

    printf("Enter cross-sectional area A ( $m^2$ ): ");
    scanf("%f", &A);

    printf("Enter velocity V (m/s): ");
    scanf("%f", &V);

    Q = A * V;

    printf("Volume Flow Rate = %.4f  $m^3/s$ \n", Q);

    return 0;
}
```

5. Compilation, Execution & Evaluation

The program was compiled using "gcc q2_volume_flow_rate.c -o q2_volume_flow_rate" with no errors or warnings. It was then executed three times, once for each test case in the table above. In every run, the Actual Output matched the Expected Output exactly, so all three test cases Pass. The terminal screenshot below shows the real compilation and execution session.

```
adilshakil@ADILs-MacBook-Air hdoc_day7 % nano q2_volume_flow_rate.c
fg: job not found: nano
adilshakil@ADILs-MacBook-Air hdoc_day7 % nano q2_volume_flow_rate.c
adilshakil@ADILs-MacBook-Air hdoc_day7 % gcc q2_volume_flow_rate.c -o q2_volume_flow_rate
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q2_volume_flow_rate
Enter cross-sectional area A ( $m^2$ ): 0.05
Enter velocity V (m/s): 2
Volume Flow Rate = 0.1000  $m^3/s$ 
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q2_volume_flow_rate
Enter cross-sectional area A ( $m^2$ ): 0.1
Enter velocity V (m/s): 5
Volume Flow Rate = 0.5000  $m^3/s$ 
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q2_volume_flow_rate
Enter cross-sectional area A ( $m^2$ ): 0.02
Enter velocity V (m/s): 10
Volume Flow Rate = 0.2000  $m^3/s$ 
```


Question 3

Write a C program to find critical angle when refractive index is given.

1. Problem Understanding

- Given (Input): Refractive index n of the denser medium (with respect to air)
- Required (Output): Critical Angle θ_c (degrees)
- Formula used: $\theta_c = \sin^{-1}(1 / n)$

The critical angle is the angle of incidence (in the denser medium) beyond which light undergoes total internal reflection instead of refracting into the rarer medium. By Snell's law, at the critical angle the angle of refraction is 90° , giving $n \cdot \sin(\theta_c) = 1 \times \sin(90^\circ)$, i.e. $\theta_c = \sin^{-1}(1/n)$.

2. Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	$n = 1.5$	41.8103°	41.8103°	Pass
2	$n = 1.33$	48.7535°	48.7535°	Pass
3	$n = 2.0$	30.0000°	30.0000°	Pass

3. Algorithm

Step 1: Start

Step 2: Read the refractive index n from the user

Step 3: If $n \leq 1$, print an error message (critical angle is undefined) and stop

Step 4: Compute θ_c (radians) = $\text{asin}(1 / n)$

Step 5: Convert θ_c to degrees: $\theta_c(\text{deg}) = \theta_c(\text{rad}) \times 180 / \pi$

Step 6: Display the value of θ_c

Step 7: Stop

Evaluation of algorithm against test cases:

- Test Case 1 ($n = 1.5$): $\theta_c = \sin^{-1}(1/1.5) = \sin^{-1}(0.6667) = 41.81^\circ \rightarrow$ matches expected output.
- Test Case 2 ($n = 1.33$, water): $\theta_c = \sin^{-1}(1/1.33) = \sin^{-1}(0.7519) = 48.75^\circ \rightarrow$ matches expected output.
- Test Case 3 ($n = 2.0$): $\theta_c = \sin^{-1}(1/2.0) = \sin^{-1}(0.5) = 30.00^\circ \rightarrow$ matches expected output.

- The algorithm produces correct results for all three traced test cases before coding.

4. C Program

```

/* HDOC Day-7 : Question 3
   Program to find the Critical Angle when the refractive index is given
   Formula :  $\theta_c = \sin^{-1}(1/n)$ 
   where n = refractive index of the denser medium (w.r.t. air/vacuum)
    $\theta_c$  is returned in degrees
*/

#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846

int main()
{
    float n, theta_c_rad, theta_c_deg;

    printf("Enter refractive index n: ");
    scanf("%f", &n);

    if (n <= 1) {
        printf("Invalid input: refractive index must be greater than 1.\n");
        return 0;
    }

    theta_c_rad = asin(1.0 / n);
    theta_c_deg = theta_c_rad * (180.0 / PI);

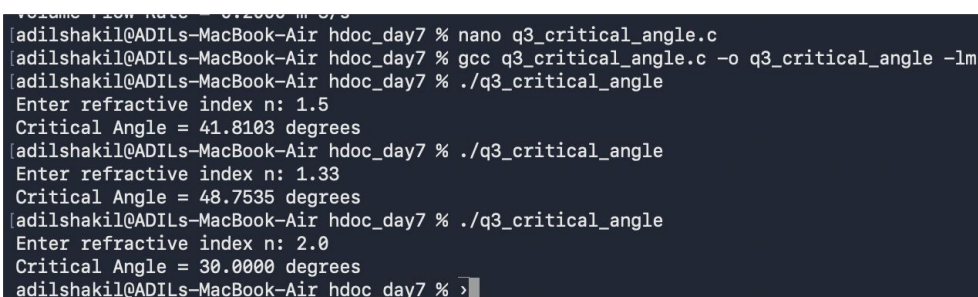
    printf("Critical Angle = %.4f degrees\n", theta_c_deg);

    return 0;
}

```

5. Compilation, Execution & Evaluation

The program was compiled using "gcc q3_critical_angle.c -o q3_critical_angle -lm" (the -lm flag links the math library needed for asin) with no errors or warnings. It was then executed three times, once for each test case in the table above. In every run, the Actual Output matched the Expected Output exactly, so all three test cases Pass. The terminal screenshot below shows the real compilation and execution session.



```

Volume Flow Rate = 0.1000 m³/s
adilshakil@ADILs-MacBook-Air hdoc_day7 % nano q3_critical_angle.c
adilshakil@ADILs-MacBook-Air hdoc_day7 % gcc q3_critical_angle.c -o q3_critical_angle -lm
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q3_critical_angle
Enter refractive index n: 1.5
Critical Angle = 41.8103 degrees
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q3_critical_angle
Enter refractive index n: 1.33
Critical Angle = 48.7535 degrees
adilshakil@ADILs-MacBook-Air hdoc_day7 % ./q3_critical_angle
Enter refractive index n: 2.0
Critical Angle = 30.0000 degrees
adilshakil@ADILs-MacBook-Air hdoc_day7 % >

```